

API and REST API Guide

Quick idea: An API lets software systems talk to each other. A REST API is a common way to design web APIs where the URL identifies the resource and the HTTP method identifies the action.

1. Introduction

In today's digital world, almost every application we use, whether it is a mobile app, website, or software, communicates with a backend server to fetch or store data.

For example:

- When you open Instagram, posts are loaded from a server
- When you order food on Swiggy, restaurant data comes from a backend
- When you search on Amazon, product data is fetched from a database

All this communication happens through something called an **API**.

2. What is an API?

API stands for **Application Programming Interface**.

In simple words, an API is a controlled way for one software system to ask another software system for data or actions. It acts like a messenger or contract between systems. A client sends a request, the server processes it, and the server sends back a response.

- Examples of API usage include a web frontend calling a backend, a mobile app logging in to a server, or a backend service calling an SMS, email, or payment service.

3. Why do we need APIs?

Modern applications are usually split into parts such as frontend, backend, database, and external services. APIs allow these parts to communicate without exposing internal implementation details.

Need	Why it matters
Separation of concerns	Frontend, backend, and database can evolve independently.
Security	Clients do not directly access the database; they use controlled endpoints.
Reusability	The same backend API can serve web, mobile, and third-party clients.
Scalability	Clear interfaces make systems easier to maintain and extend.

4. What is a REST API?

REST stands for **Representational State Transfer**. A REST API is a web API designed around resources such as tasks, users, orders, or students.

REST focuses on predictable, resource-based URLs. Instead of putting the action in the URL, REST usually puts the action in the HTTP method.

Core REST idea: the URL identifies the resource, and the HTTP method tells the server what to do with that resource.

Architecture

Client (UI / Postman)



Controller



Service



Repository



Database

5. Resource-based design

A resource is a meaningful business object or collection of objects. In a Todo application, 'tasks' is a resource.

Resource form	Meaning
/api/tasks	The tasks collection
/api/tasks/15	A single task with id 15
/api/users	The users collection
/api/users/7	A single user with id 7

6. HTTP methods used in REST APIs

Method	Typical use	Example	Notes
GET	Fetch data	GET /api/tasks	Does not change server state.
POST	Create a new resource	POST /api/tasks	Body usually contains new data.
PUT	Replace or fully update a resource	PUT /api/tasks/15	Often used for updates.
PATCH	Partially update a resource	PATCH /api/tasks/15	Updates selected fields.
DELETE	Remove a resource	DELETE /api/tasks/15	May return 204 or a message.

7. Common status codes

Status code	Name	Meaning
200	OK	Success.
201	Created	Resource created.
400	Bad Request	Invalid input.
401	Unauthorized	Auth required or invalid.
404	Not Found	Resource does not exist.
500	Internal Server Error	Unexpected server error.

8. Standard REST URL pattern

In REST-style APIs, the path usually stays the same for the same resource, while the HTTP method changes based on the operation.

Endpoint	Meaning
GET /api/tasks	Fetch the task collection
POST /api/tasks	Create a new task in the collection
GET /api/tasks/{taskId}	Fetch one task
PUT /api/tasks/{taskId}	Replace or fully update a task
PATCH /api/tasks/{taskId}	Partially update a task
DELETE /api/tasks/{taskId}	Delete a task

9. Why this design is preferred

- It keeps APIs predictable: developers can guess endpoints from standard conventions.
- It matches HTTP semantics: GET reads, POST creates, PUT/PATCH update, and DELETE removes.
- It works well with common tools such as browsers, mobile apps, Postman, Swagger, and frontend frameworks.
- It scales cleanly as the API grows to include resources such as users, products, orders, or enrollments.

10. Request and response flow

- The user performs an action in the UI, such as opening a task list or submitting a form.
- The frontend sends an HTTP request to the backend API.
- The controller receives the request and maps it to the correct endpoint.
- The service layer validates the request and applies business rules.
- The repository or data access layer reads or writes data in the database.
- The backend returns a response, often in JSON format, with an HTTP status code.

11. Practical Todo example

Imagine a Todo application with a frontend, a Spring Boot backend, and a MySQL database.

User action	API call	Expected result
Open task list	GET /api/tasks	List of tasks is returned.
Create a task	POST /api/tasks	A new task is saved and returned.
Open one task	GET /api/tasks/7	Task 7 details are returned.
Edit a task	PUT /api/tasks/7	Task 7 is updated.
Mark completed only	PATCH /api/tasks/7	Only selected fields are changed.

Delete a task	DELETE /api/tasks/7	Task 7 is removed.
---------------	---------------------	--------------------

12. Example JSON payloads

Create request body:

```
{
  "title": "Record REST API video",
  "completed": false
}
```

Typical response body:

```
{
  "id": 7,
  "title": "Record REST API video",
  "completed": false
}
```

13. Key beginner takeaways

- API = a communication contract between systems.
- REST API = a resource-based way to design web APIs.
- URL = which resource you are talking about.
- HTTP method = what you want to do with that resource.
- Use nouns in URLs and actions in methods.
- Prefer /api/tasks over /getTasks for standard REST design.